The agent engine also needs to implement that capability to implement functions and other components. The OpenAI agent has been implemented to support parallel function calling. Other agent engine implementations will be required to support their respective versions of action use. Fortunately, the definition of the OpenAI tool is becoming the standard, and many platforms adhere to this standard.

Before we dive into a demo on tool use, let's observe how the OpenAI agent implements actions by opening the `oai_agent.py` file in VS Code. The following listing shows the top of the agent's `get_response_stream` function and its implementation of function calling.

Listing 7.13 Caling the API in `get_response_stream`

```
def get_response_stream(self, user_input, thread_id=None):
    self.last_message = ""
    self.messages += [{"role": "user", "content": user_input}]
    if self.tools and len(self.tools) > 0:    #1
        response = self.client.chat.completions.create(
            model=self.model,
            messages=self.messages,
            tools=self.tools,      #2
            tool_choice="auto",      #3
        )
    else:     #4
        response = self.client.chat.completions.create(
            model=self.model,
            messages=self.messages,
        )
    response_message = response.choices[0].message
    tool_calls = response_message.tool_calls     #5
```

#1 Detects whether the agent has any available tools turned on
#2 Sets the tools in the chat completions call
#3 Ensures that the LLM knows it can choose any tool
#4 If no tools, calls the LLM the standard way
#5 Detects whether there were any tools used by the LLM

Executing the functions follows, as shown in listing 7.14. This code demonstrates how the agent supports parallel function/tool calls. These calls are parallel because the agent executes each one together and in no order. In chapter 11, we'll look at planners that allow actions to be called in ordered sequences.